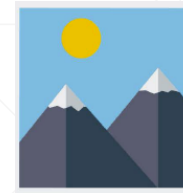




开发基于像素的绘图App

北京理工大学计算机学院
金旭亮

两种类型的绘图App



矢量绘图App

比如Visio, Word中的
绘图功能

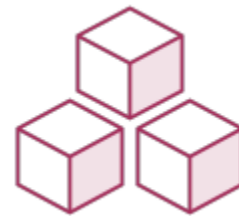
提供一些标准的可以使用数学方式描述的图形，支持快速的重绘与无精度损失的缩放，文件占用空间少。缺点在于细节缺失和不够逼真。

基于像素的绘图App

比如Photoshop

直接以像素为基本单位处理位图，处理起来比较耗费系统资源，高精度图像文件占用空间多，图像放大后往往会有精度损失，但细节丰富逼真，多来自于数码相机的现场拍摄。

JavaFX中的绘图组件

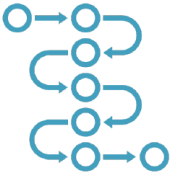


- ✓ JavaFX所提供的Shape组件，主要用于实现矢量绘图。
- ✓ JavaFX同样支持基于像素的绘图模式，主要由以下两个组件实现：
 - **Canvas**：代表一个绘图表面，本质上是一个bitmap对象，支持特效和变换。
 - **GraphicsContext**：提供一组绘图命令，在Canvas上绘图。



当Canvas加入到Scene Graph之后，只能在主线程（JavaFX Application Thread）中绘图。

绘图步骤



- ▶ 首先需要new一个Canvas对象，指定它的宽度和高度，它设定了绘图范围：

```
Canvas canvas = new Canvas(500, 300);
```

- ▶ 接着，将Canvas对象加入到控件树中：

```
root.getChildren().add(canvas);
```

- ▶ 然后，可以获取绘图上下文：

```
GraphicsContext gc = canvas.getGraphicsContext2D();
```

- ▶ 现在，可以基于GraphicsContext对象所提供的方法，完成许多基础绘图功能（比如，绘制矩形和圆）。

二维绘图的基本设置



在绘图前，通常会进行一些设置，比如设定背景色，指定要使用的Fill和Stroke，之后再调用相应的绘图方法。

```
GraphicsContext gc = canvas.getGraphicsContext2D();  
gc.setStroke(Color.RED);  
gc.setFill(Color.BLUE);  
gc.strokeLine(10, 50, 50, 10);  
gc.fillOval(60, 10, 80, 80);
```



可以给root节点附加样式，从而影响到作为其子节点的Canvas相关属性。

二维绘图示例



示例中的顶层容器是VBox，里面放置了一个Canvas作为绘图面板。

示例中使用GraphicsContext对象所提供的绘图指令，绘制了一幅图片，一条分隔线，还有一段文本作为图片说明。

示例分析

```
private void drawOnCanvas(Canvas canvas) {  
    //获取绘图上下文  
    GraphicsContext gc = canvas.getGraphicsContext2D();  
    //从资源文件中读取图片  
    String image = getClass().getResource(name: "lake.jpg").toExternalForm();  
    Image moon = new Image(image);  
    //在Canvas的左上角, 按照宽600像素, 高379像素绘图, 这实际上就是图片的原始分辨率  
    gc.drawImage(moon, x: 0, y: 0, w: 600, h: 379);  
    //使用宽度为4像素的蓝色线条, 绘制一条分割线  
    gc.setStroke(Color.BLUE);  
    gc.setLineWidth(4d);  
    gc.strokeLine(x1: 0, y1: 390, x2: 600, y2: 390);  
    //在 (250, 435) 处绘制图片标题  
    gc.setFont(new Font(size: 40));  
    gc.setFill(Color.CADETBLUE);  
    gc.fillText(text: "湖光山色", x: 250, y: 435);  
}
```

练习：绘制简单图形



GraphicsContext提供了方法绘制简单的图形，请参考GraphicsContext文档，使用其方法绘制左侧的笑脸。